

API REST CON SPRING BOOT Y JWT

Documentacion Tecnica y Manual de Endpoints

Soluciones Web y Aplicaciones Distribuidas
Ing. CRISÓLOGO BARDALES Percy
Junio 2026

Spring Boot 3.3.5 | Java 21 | JWT | H2 Database

1. Introduccion y Conceptos

1.1 Que es una API REST?

Una API REST (Representational State Transfer) es un estilo de arquitectura de software que define un conjunto de principios para disenar servicios web. Permite la comunicacion entre sistemas mediante el protocolo HTTP, usando verbos como GET, POST, PUT y DELETE para representar operaciones sobre recursos. Cada recurso se identifica mediante una URL unica y las respuestas se devuelven generalmente en formato JSON.

Principios REST:

- Sin estado (Stateless): cada solicitud contiene toda la informacion necesaria para ser procesada.
- Interfaz uniforme: los recursos se identifican y manipulan de manera consistente.
- Sistema en capas: permite agregar intermediarios como proxies o balanceadores de carga.
- Cache: las respuestas pueden almacenarse en cache para mejorar el rendimiento.

1.2 Que es Spring Boot?

Spring Boot es un framework de Java basado en Spring Framework que simplifica la creacion de aplicaciones empresariales. Ofrece configuracion automatica (auto-configuration), un servidor embebido (Tomcat por defecto), y un sistema de dependencias gestionado. Permite desarrollar APIs REST de forma rapida sin necesidad de XML de configuracion.

Caracteristicas principales:

- Configuracion automatica mediante anotaciones (@SpringBootApplication, @RestController, etc.).
- Servidor embebido Tomcat/Jetty incluido en el JAR final.
- Spring Data JPA para acceso a base de datos con Hibernate.
- Spring Security para autentificacion y autorizacion robusta.
- Bean Validation (Jakarta) para validar entradas de datos del cliente.

1.3 Que es JWT (JSON Web Token)?

JWT es un estandar abierto (RFC 7519) que define un formato compacto y autocontenido para transmitir informacion entre partes como un objeto JSON. Los tokens estan firmados digitalmente con HMAC o RSA, lo que garantiza su autenticidad e integridad sin necesidad de consultar la base de datos en cada peticion.

Estructura de un JWT (tres partes separadas por punto):

- Header: tipo de token y algoritmo de firma (ej: HS256).
- Payload: claims o afirmaciones (sub, iat, exp, rol, etc.).
- Signature: firma digital para verificar la integridad del token.

Ejemplo de token JWT:

```
eyJhbGciOiJIUzI1NiJ9
.eyJzdWIiOiIxMjMONTY3OCIsInJvbCI6IkFETU1OIiwiaWF0IjoxNzE3MDAwMDAwfQ
.abcl23signatureHere
```

Flujo de autenticacion JWT:

1. El cliente envia sus credenciales (DNI + contraseña) al endpoint POST /api/auth/login.
2. El servidor valida las credenciales contra la base de datos usando BCrypt.
3. Si son correctas, genera un JWT firmado y lo retorna al cliente.
4. El cliente almacena el token (localStorage, memoria, etc.).
5. En cada solicitud posterior el cliente envia: Authorization: Bearer <token>.
6. El JwtFilter intercepta la peticion y valida la firma y expiracion del token.
7. Si el token es valido, el request se procesa normalmente; si no, se rechaza con 403.

1.4 Arquitectura en Capas del Proyecto

El proyecto sigue el patron de capas (Layered Architecture) de Spring:

- Model: entidades JPA anotadas con @Entity que representan tablas de la base de datos.
- Repository: interfaces que extienden JpaRepository; Spring Data genera las consultas automaticamente.
- Service: logica de negocio. Intermediario entre controllers y repositories.
- DTO (Data Transfer Objects): clases para transportar datos entre capas sin exponer el modelo directamente.
- Controller: reciben peticiones HTTP, validan entradas con @Valid y llaman al servicio.
- Security: JwtUtil, JwtFilter y UserDetailsServiceImpl gestionan la autenticacion JWT.
- Config: SecurityConfig define reglas de acceso y configura la cadena de filtros.

1.5 Relaciones entre Modelos JPA

El proyecto implementa dos tipos de relaciones JPA para demostrar cardinalidades:

OneToMany — Una marca tiene muchos vehiculos:

- La entidad Marca contiene una List<Vehiculo> anotada con @OneToMany(mappedBy="marca").
- mappedBy indica que Vehiculo es el dueño de la relacion (tiene la clave foranea).
- cascade=ALL propaga operaciones CRUD al hijo; fetch=LAZY para cargar solo cuando se necesite.

ManyToOne — Muchos vehiculos pertenecen a una marca:

- La entidad Vehiculo tiene un campo Marca anotado con @ManyToOne.
- @JoinColumn(name="marca_id") define la columna de clave foranea en la tabla vehiculo.
- fetch=EAGER carga la marca junto con el vehiculo automaticamente.

1.6 Validaciones con Bean Validation

Spring Boot integra Jakarta Bean Validation para validar datos de entrada en DTOs y entidades:

- @NotBlank: el campo no puede ser nulo ni cadena vacia (aplica a String).
- @NotNull: el campo no puede ser nulo (aplica a cualquier tipo).
- @Size(min=6): define longitud minima de 6 caracteres.
- @Email: valida formato de correo electronico.

Las anotaciones se activan en los controladores con @Valid en el parametro @RequestBody. Cuando falla la validacion, Spring retorna automaticamente HTTP 400 con los mensajes de error configurados.

2. Estructura del Proyecto

2.1 Arbol de Directorios

```
spjwt/
├── pom.xml
├── src/main/
│   ├── java/com/example/spjwt/
│   │   ├── SpjwtApplication.java      (Clase principal)
│   │   ├── config/
│   │   │   └── SecurityConfig.java    (Spring Security)
│   │   ├── controller/
│   │   │   ├── AuthController.java   (Login y registro)
│   │   │   ├── VehiculoController.java (CRUD vehiculos)
│   │   │   └── MarcaController.java   (CRUD marcas)
│   │   ├── dto/
│   │   │   ├── LoginRequest.java
│   │   │   ├── LoginResponse.java
│   │   │   ├── VehiculoDto.java
│   │   │   └── MarcaDto.java
│   │   ├── model/
│   │   │   ├── Usuario.java
│   │   │   ├── Vehiculo.java
│   │   │   └── Marca.java
│   │   ├── repository/
│   │   │   ├── UsuarioRepository.java
│   │   │   ├── VehiculoRepository.java
│   │   │   └── MarcaRepository.java
│   │   ├── security/
│   │   │   ├── JwtUtil.java
│   │   │   ├── JwtFilter.java
│   │   │   └── UserDetailsServiceImpl.java
│   │   └── service/
│   │       ├── AuthService.java
│   │       ├── VehiculoService.java
│   │       └── MarcaService.java
│   └── resources/
│       ├── application.properties
│       └── data.sql
```

2.2 Dependencias Principales (pom.xml)

Dependencia	Proposito
spring-boot-starter-web	API REST con Tomcat embebido
spring-boot-starter-security	Framework de seguridad
spring-boot-starter-data-jpa	Acceso a BD con Hibernate/JPA
spring-boot-starter-validation	Validacion Bean Validation
h2	Base de datos en memoria para pruebas

Dependencia	Proposito
jjwt-api / jjwt-impl / jjwt-jackson	Generacion y validacion JWT (v0.12.6)

3. Usuarios de Prueba

La base de datos H2 se inicializa con data.sql al arrancar la aplicacion. Contiene dos usuarios con contraseñas encriptadas en BCrypt:

Campo	Usuario Admin	Usuario Normal	Descripcion
DNI	12345678	87654321	Nombre de usuario para login
Contraseña	admin123	user1234	Minimo 6 caracteres
Rol	ADMIN	USER	Rol asignado al usuario

La contraseña se almacena como hash BCrypt. Nunca se guarda en texto plano.

4. Endpoints de la API

Base URL: `http://localhost:8080`

Formato de respuestas: JSON | Autenticacion: Bearer JWT

4.1 Autenticacion — No requieren JWT

POST `/api/auth/login` — Iniciar Sesión

Autentica al usuario con DNI y contraseña. Si las credenciales son validas retorna un token JWT firmado con expiracion de 1 hora.

Headers / Notas:

- Content-Type: application/json

Request Body:

```
{ "dni": "12345678", "password": "admin123" }
```

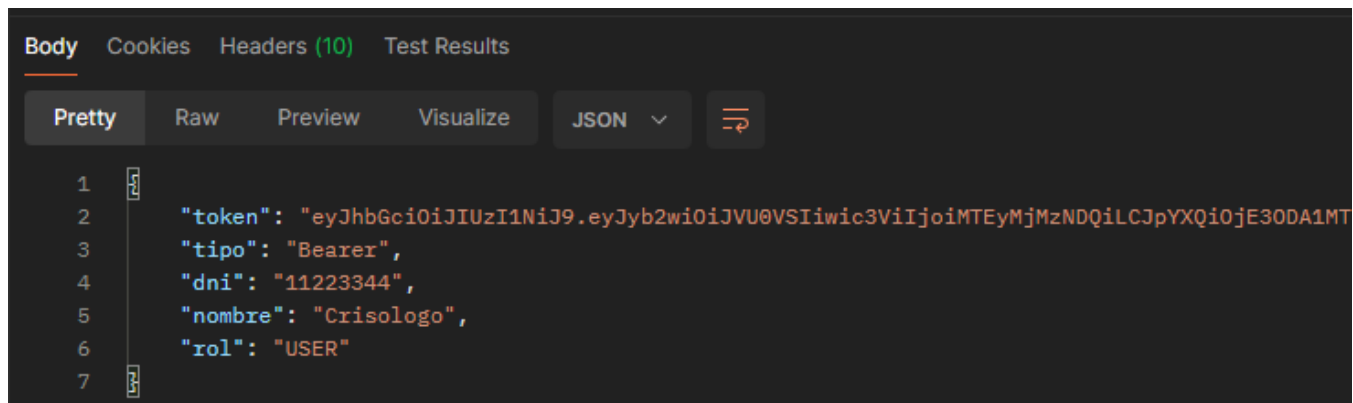
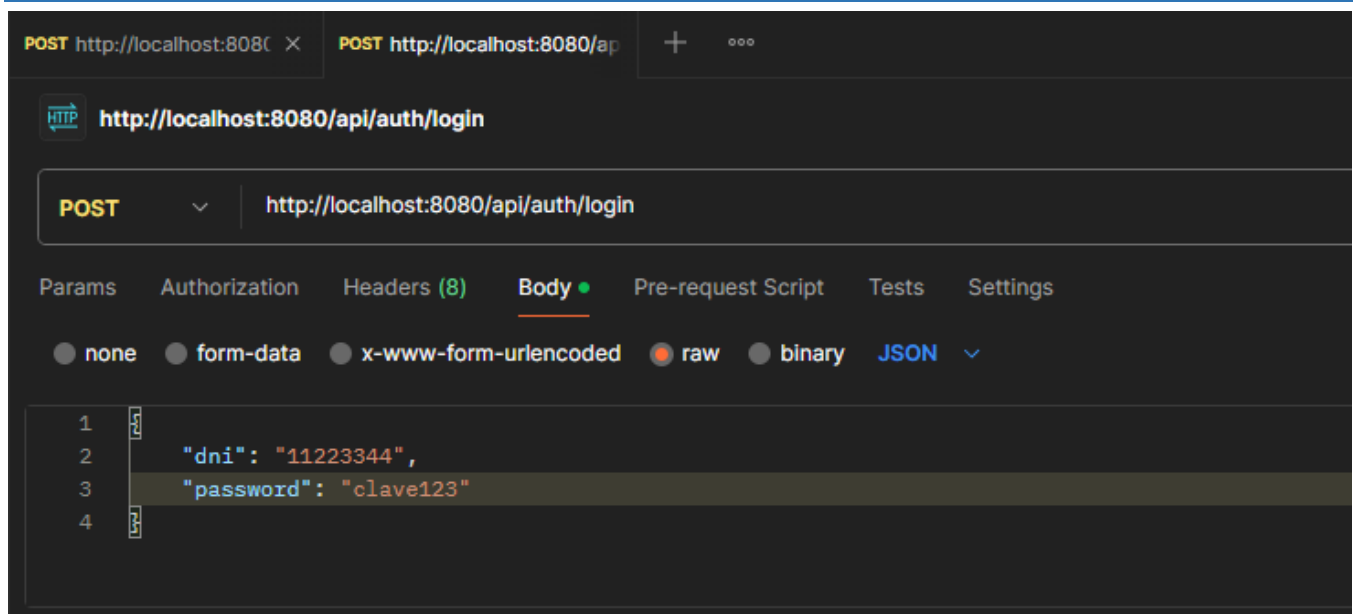
Respuestas:

200 OK { "token": "eyJ...", "tipo": "Bearer", "dni": "12345678", "nombre": "Administrador Sistema", "rol": "ADMIN" }

401 Unauthorized "Usuario no existe"

401 Unauthorized "Credenciales incorrectas: DNI o contraseña invalidos"

Ejemplo en Postman



POST /api/auth/registro — Registrar Usuario

Registra un nuevo usuario en el sistema. La contraseña se encripta con BCrypt antes de persistirse. El campo password en la respuesta se muestra como [PROTEGIDO].

Headers / Notas:

- Content-Type: application/json

Request Body:

```

{
  "dni": "11223344",
  "nombre": "Maria Lopez",
  "password": "clave123",
  "rol": "USER"
}

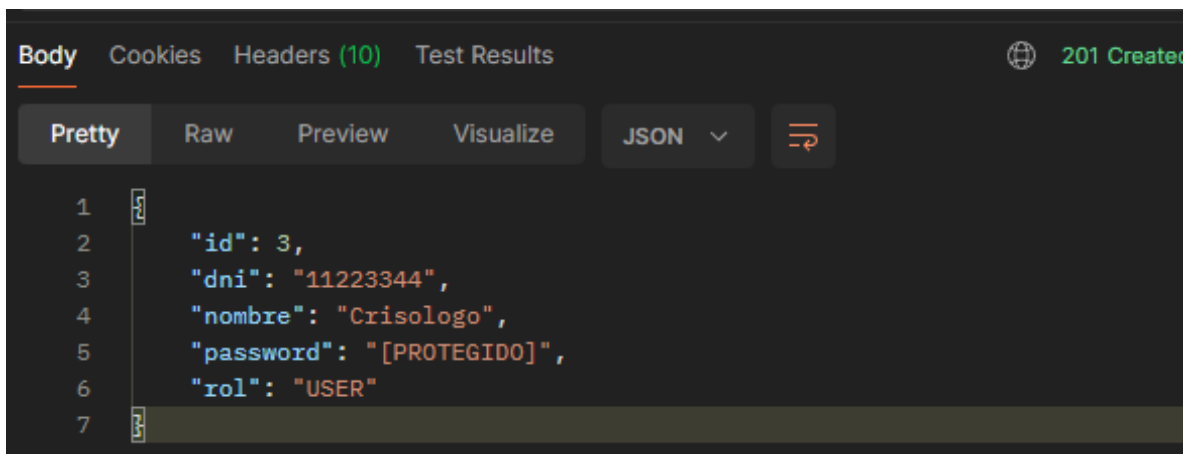
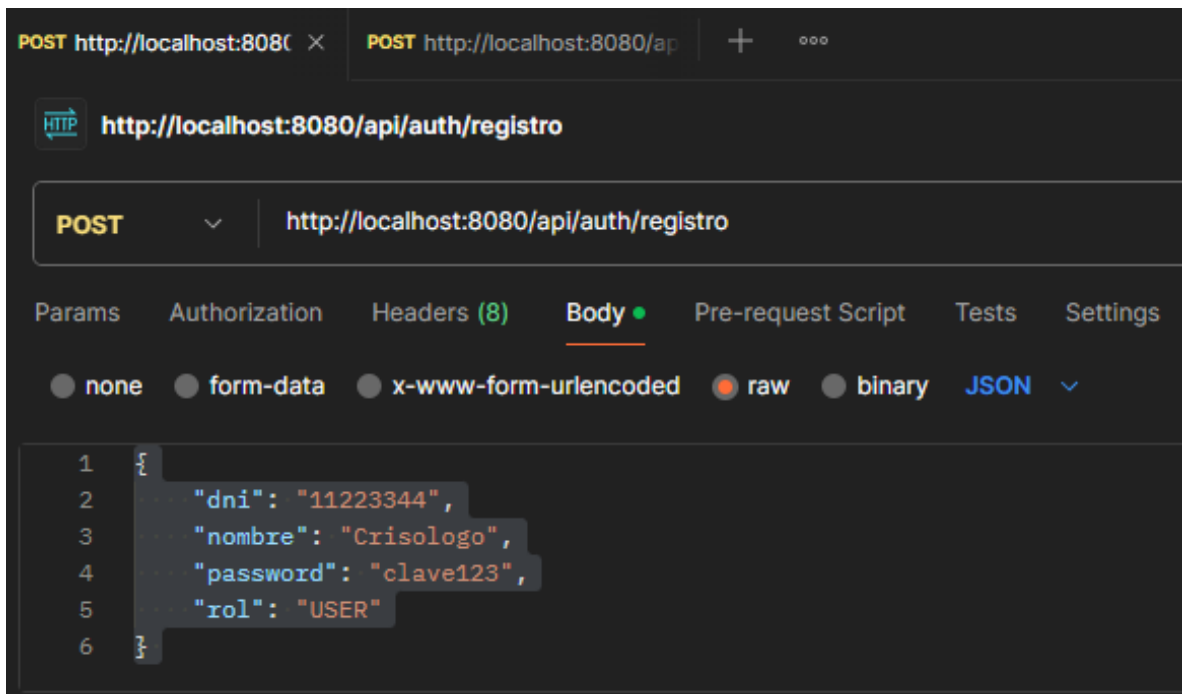
```

Respuestas:

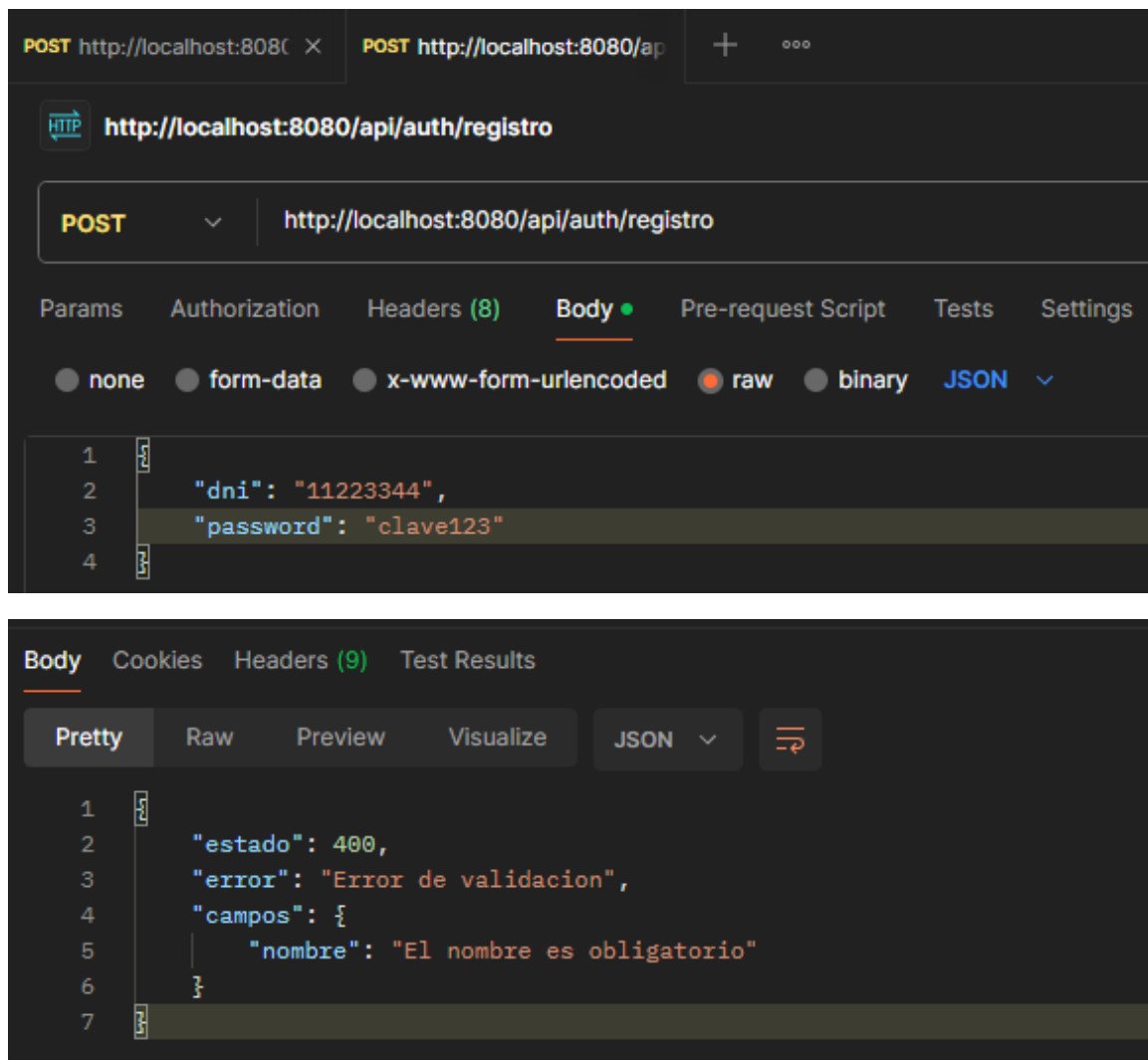
201 Created { "id": 3, "dni": "11223344", "nombre": "Maria Lopez", "password": "[PROTEGIDO]", "rol": "USER" }

400 Bad Request "Ya existe un usuario con el DNI: 11223344"

Ejemplo en Postman



Validación en caso de que no se envíen todos los datos



The image shows two screenshots of a REST client interface. The top screenshot displays a POST request to `http://localhost:8080/api/auth/registro` with a JSON body: `{ "dni": "11223344", "password": "clave123" }`. The bottom screenshot shows the response body, which is a JSON object: `{ "estado": 400, "error": "Error de validacion", "campos": { "nombre": "El nombre es obligatorio" } }`.

4.2 Vehiculos — Requieren JWT

IMPORTANTE: Todos los endpoints `/api/vehiculos` requieren el header: `Authorization: Bearer <token>`. Sin token valido la respuesta es HTTP 403 Forbidden.

GET /api/vehiculos — *Listar todos los vehiculos*

Retorna la lista completa de vehiculos registrados. Incluye el nombre de la marca (relacion ManyToOne).

Headers / Notas:

- Authorization: Bearer <token>

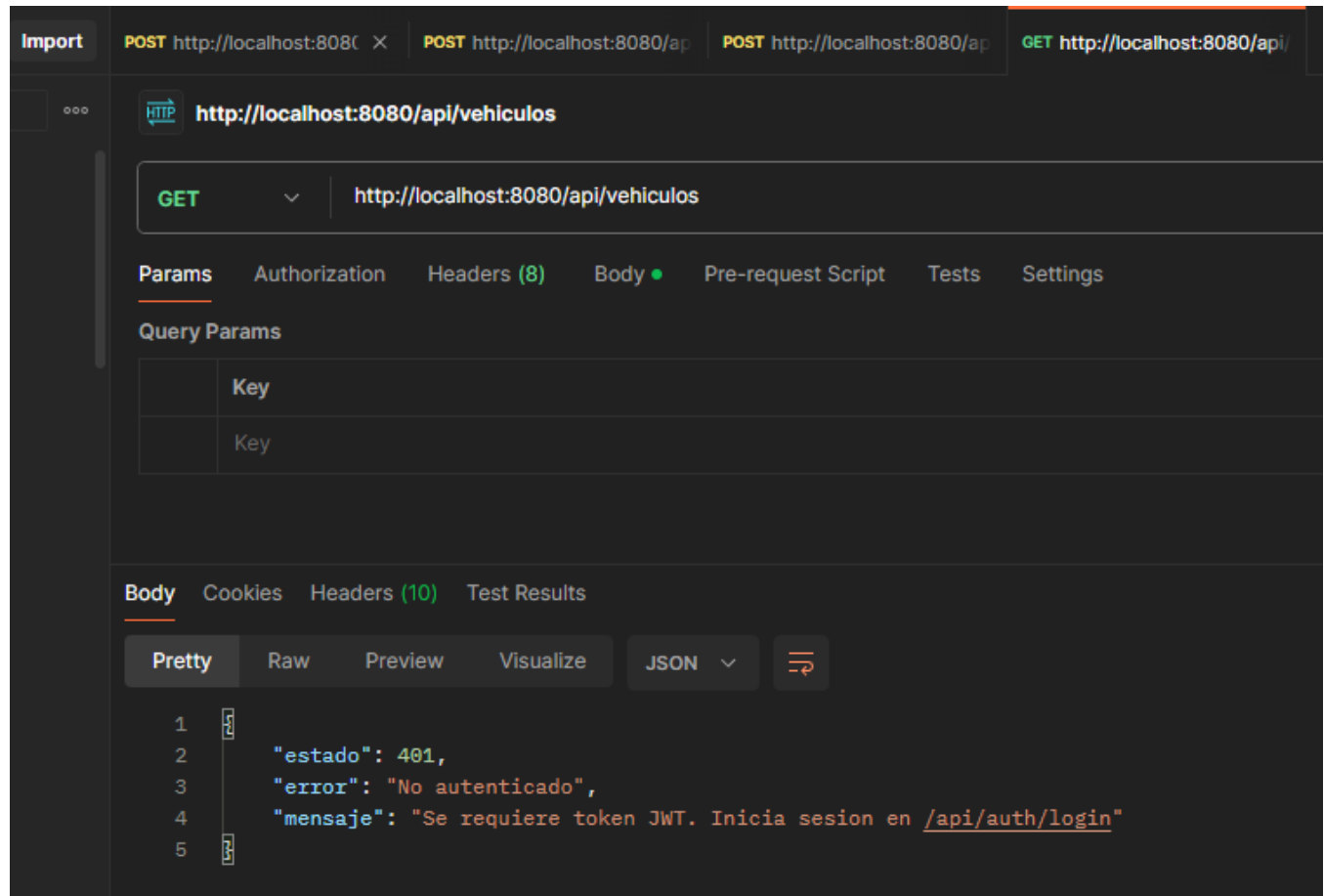
Respuestas:

200 OK [{ "id": 1, "modelo": "Corolla", "anio": 2022, "placa": "ABC-123", "marcaId": 1, "marcaNombre": "Toyota" }, ...]

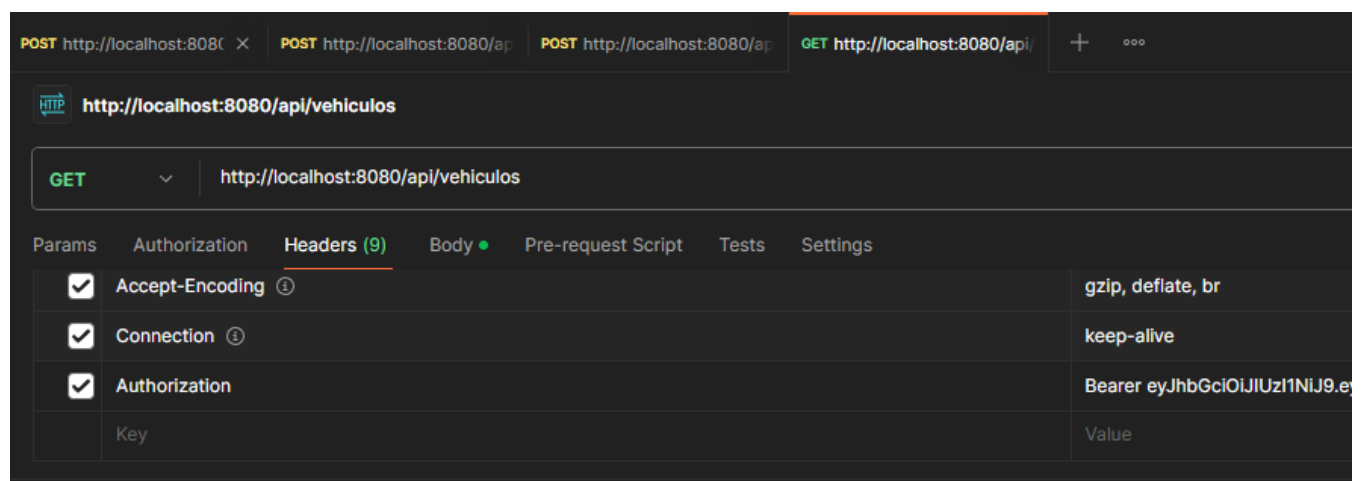
403 Forbidden { "status": 403, "error": "Forbidden", "message": "Access Denied" }

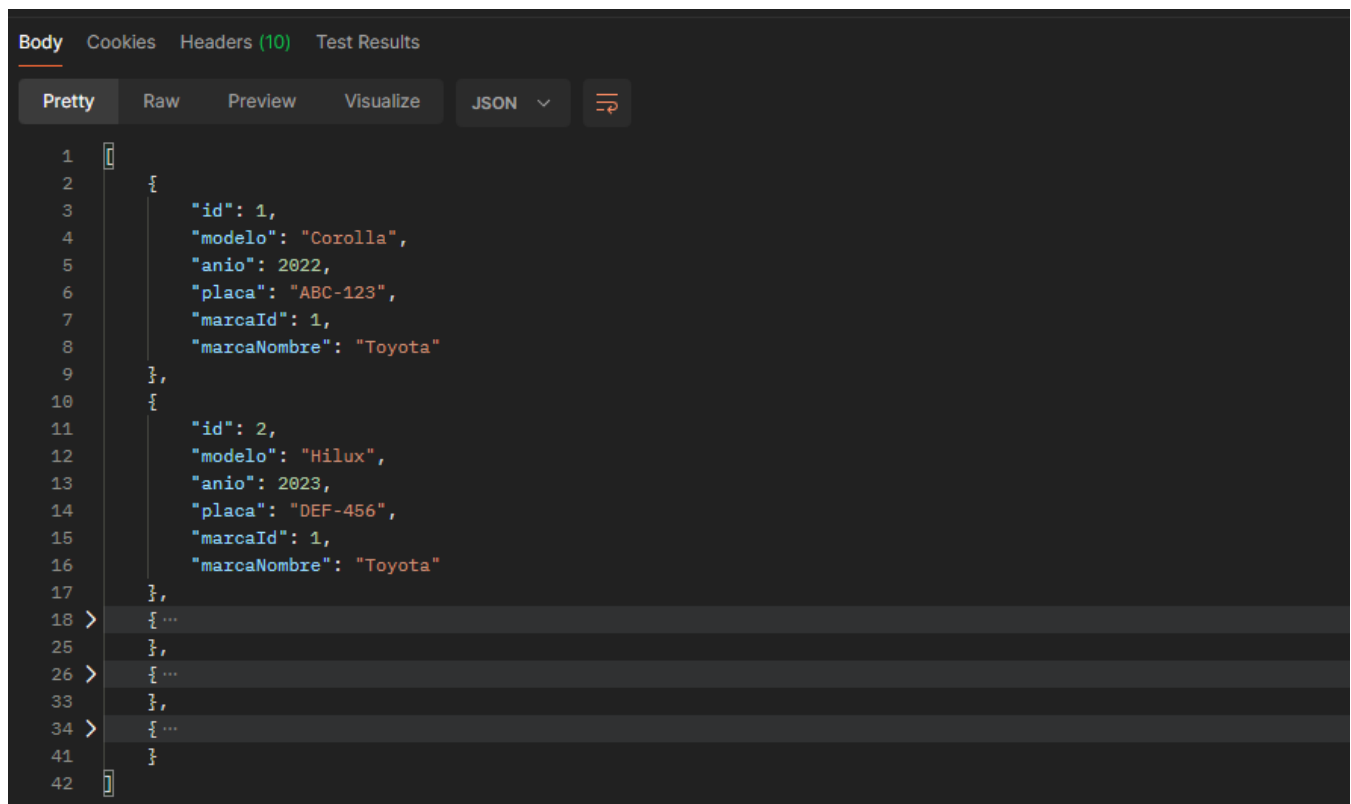
Ejemplo en Postman

Si no se enviamos el token devuelve error



Ejemplo de como enviar el token en cada petición





The screenshot shows a REST client interface with tabs for Body, Cookies, Headers (10), and Test Results. The Body tab is active, displaying a JSON response in 'Pretty' format. The JSON contains an array of two vehicle objects. The first object has id: 1, modelo: 'Corolla', anio: 2022, placa: 'ABC-123', marcaId: 1, and marcaNombre: 'Toyota'. The second object has id: 2, modelo: 'Hilux', anio: 2023, placa: 'DEF-456', marcaId: 1, and marcaNombre: 'Toyota'. The interface includes a 'Raw' tab, a 'Visualize' button, a 'JSON' dropdown, and a refresh icon.

```
1 [
2   {
3     "id": 1,
4     "modelo": "Corolla",
5     "anio": 2022,
6     "placa": "ABC-123",
7     "marcaId": 1,
8     "marcaNombre": "Toyota"
9   },
10  {
11    "id": 2,
12    "modelo": "Hilux",
13    "anio": 2023,
14    "placa": "DEF-456",
15    "marcaId": 1,
16    "marcaNombre": "Toyota"
17  },
18  ...
25  },
26  ...
33  },
34  ...
41  }
42 ]
```

GET /api/vehiculos/{id} — Buscar vehiculo por ID

Retorna el detalle de un vehiculo especifico por su identificador numerico.

Headers / Notas:

- Authorization: Bearer <token>
- Ejemplo: GET /api/vehiculos/1

Respuestas:

200 OK { "id": 1, "modelo": "Corolla", "anio": 2022, "placa": "ABC-123", "marcaId": 1, "marcaNombre": "Toyota" }

404 Not Found "Vehiculo con ID 99 no encontrado"

GET /api/vehiculos/marca/{marcaId} — Listar vehiculos por marca

Filtra y retorna todos los vehiculos de una marca especifica. Demuestra la relacion OneToMany: una marca tiene muchos vehiculos.

Headers / Notas:

- Authorization: Bearer <token>
- Ejemplo: GET /api/vehiculos/marca/1 (todos los Toyota)

Respuestas:

200 OK [{ "id": 1, "modelo": "Corolla", ... }, { "id": 2, "modelo": "Hilux", ... }]

403 Forbidden Sin token valido

POST /api/vehiculos — *Crear vehiculo*

Registra un nuevo vehiculo. La marca referenciada debe existir. La placa no puede estar duplicada.

Headers / Notas:

- Authorization: Bearer <token>
- Content-Type: application/json

Request Body:

```
{
  "modelo": "Camry",
  "anio": 2024,
  "placa": "PQR-789",
  "marcaId": 1
}
```

Respuestas:

201 Created { "id": 6, "modelo": "Camry", "anio": 2024, "placa": "PQR-789", "marcaId": 1, "marcaNombre": "Toyota" }

400 Bad Request "No existe una marca con ID: 99"

400 Bad Request "Ya existe un vehiculo con la placa: PQR-789"

DELETE /api/vehiculos/{id} — *Eliminar vehiculo*

Elimina un vehiculo de la base de datos por su ID.

Headers / Notas:

- Authorization: Bearer <token>
- Ejemplo: DELETE /api/vehiculos/1

Respuestas:

200 OK "Vehiculo eliminado correctamente"

404 Not Found "Vehiculo con ID 1 no encontrado"

4.3 Marcas — Requieren JWT

GET /api/marcas — *Listar todas las marcas*

Retorna la lista de marcas con el contador de vehiculos asociados a cada una (campo totalVehiculos).

Headers / Notas:

- Authorization: Bearer <token>

Respuestas:

200 OK [{ "id": 1, "nombre": "Toyota", "paisOrigen": "Japon", "totalVehiculos": 2 }, ...]

GET /api/marcas/{id} — *Buscar marca por ID*

Retorna el detalle de una marca especifica.

Headers / Notas:

- Authorization: Bearer <token>
- Ejemplo: GET /api/marcas/1

Respuestas:

200 OK { "id": 1, "nombre": "Toyota", "paisOrigen": "Japon", "totalVehiculos": 2 }

404 Not Found "Marca con ID 99 no encontrada"

POST /api/marcas — Crear marca

Registra una nueva marca de vehiculo en el sistema.

Headers / Notas:

- Authorization: Bearer <token>
- Content-Type: application/json

Request Body:

```
{ "nombre": "Honda", "paisOrigen": "Japon" }
```

Respuestas:

201 Created { "id": 4, "nombre": "Honda", "paisOrigen": "Japon", "totalVehiculos": 0 }

DELETE /api/marcas/{id} — Eliminar marca

Elimina una marca por su ID. Ejemplo: DELETE /api/marcas/4

Headers / Notas:

- Authorization: Bearer <token>

Respuestas:

200 OK "Marca eliminada correctamente"

404 Not Found "Marca con ID 4 no encontrada"

5. Resumen de Endpoints

Metodo	URL	Auth	Descripcion
POST	/api/auth/login	No	Iniciar sesion con DNI y contraseña
POST	/api/auth/registro	No	Registrar nuevo usuario
GET	/api/vehiculos	JWT	Listar todos los vehiculos
GET	/api/vehiculos/{id}	JWT	Buscar vehiculo por ID
GET	/api/vehiculos/marca/{id}	JWT	Listar vehiculos por marca
POST	/api/vehiculos	JWT	Crear nuevo vehiculo

Metodo	URL	Auth	Descripcion
DELETE	/api/vehiculos/{id}	JWT	Eliminar vehiculo
GET	/api/marcas	JWT	Listar todas las marcas
GET	/api/marcas/{id}	JWT	Buscar marca por ID
POST	/api/marcas	JWT	Crear nueva marca
DELETE	/api/marcas/{id}	JWT	Eliminar marca

6. Guia de Uso Rapido

6.1 Iniciar la Aplicacion

Requisitos: Java 21 y Maven 3.x instalados.

```
# Desde el directorio raiz del proyecto:  
mvn spring-boot:run
```

```
# API disponible en:  
http://localhost:8080
```

```
# Consola H2 (base de datos):  
http://localhost:8080/h2-console
```

7. Configuracion de Base de Datos H2

La aplicacion usa H2 como base de datos en memoria para desarrollo. Los datos se cargan desde data.sql al arrancar y se pierden al detener la aplicacion.

Parametro	Valor
URL Consola	http://localhost:8080/h2-console
JDBC URL	jdbc:h2:mem:spjwtodb
Usuario	sa
Contraseña	(dejar vacio)
Modo	In-memory (create-drop)

Acceso a la Base de Datos

←↻🔍localhost:8080/h2-console/login.jsp?jsessionid=a0337c4d53a74da5bd

Español⌵PreferenciasToolsAyuda

Registrar

Configuraciones guardadas:Generic H2 (Embedded)⌵

Nombre de la configuración:Generic H2 (Embedded)GuardarEliminar

Controlador:org.h2.Driver

URL JDBC:jdbc:h2:mem:spjwtdb

Nombre de usuario:sa

Contraseña:

ConectarProbar la conexión

Mostrando listado de vehículos

←↻🔍localhost:8080/h2-console/login.do?jsessionid=a0337c4d53a74da5bd83b9944b61d1af

🎵🔍🔗🔗🔗🔗🔗

🔗Auto commit🔗Número máximo de filas:1000⌵🔗🔗🔗🔗

🔗Auto completado

🔗jdbc:h2:mem:spjwtdb

🔗MARCA

- 🔗ID
- 🔗NOMBRE
- 🔗PAIS_ORIGEN
- 🔗Índices

🔗USUARIO

🔗VEHICULO

🔗INFORMATION_SCHEMA

🔗Usuarios

- 🔗SA

🔗H2 2.3.232 (2024-08-11)

EjecutarRun SelectedAuto completadoEliminarInstrucción SQL:

SELECT * FROM VEHICULO

SELECT * FROM VEHICULO;

ANIO	ID	MARCA_ID	PLACA	MODELO
2022	1	1	ABC-123	Corolla
2023	2	1	DEF-456	Hilux
2021	3	2	GHI-789	Mustang
2020	4	3	JKL-012	Golf
2023	5	3	MNO-345	Tiguan

(5 filas, 1 ms)

Editar